

Carl Chang:

For day 1:

My part here was about why it matters to present the system configurations with the performance results. First I discussed about the main idea about that:

Measuring the performance of a program and presenting its results is a really important thing. The key thing here is that the correctness of the result is not the only thing that matters. Being efficient is also really important.

It's important to present the performance result in the correct way. The performance results may depend on many external factors that are related to the hardware, software and the system. Only having results as numbers without any meaningful context is useless. For example, if you have a result like "runtime = 1 minute", the slowness of the result here can be meaningless as it can be the result of hardware, input size or compiler. If the performance result and configuration weren't presented in the correct way then there's no way to know what is causing the problem with the performance.

Then I discussed about the 3 types of configurations, their key components and how they would affect the performance results:

There are 3 types of configurations, which are hardware, software and system.

The hardware configuration is the most important of the 3 of them as it directly influences the performance of the system. The key parts about hardware configuration are CPU, clock speed, core number, cache and RAM. CPU executes the instructions such as math operations and memory accesses. It's basically the heart of things and determines how many instructions can be executed per second. The clock speed means the cycles per amount. For example, 3 GHz CPU means it could execute 3 billion cycles per second. The faster it gets, the faster it executes the instructions and the less time it takes. The core number is really important for parallel programming. The number of it means how many parts of the workloads can be divided to and executed simultaneously. For example, having 8 cores means each of it can do 1/8 of the total work. The more it gets the less time it takes. Cache is important for reducing memory access latency. It works as a temporary storage for frequently accessed data. Data stored in it can be accessed significantly faster than data in main memory. RAM stores program data. It ensures that

programs can operate without relying on much slower disk storage. It determines how much data and how many programs the system can keep readily accessible for the CPU. In general, the hardware configuration defines the performance capability of the system. It must always be reported to ensure fair comparison and reproducibility.

The software configuration determines how efficiently the hardware resources are utilized, making it also an important factor for presenting performance. The key parts about software configuration are compiler optimization, operating system and libraries. Compiler converts codes into machine instructions. Proper optimizations can drastically decrease the execution time through reducing unnecessary operations, improving instruction selection, and minimizing memory access. Operating systems play an important role through managing memory, scheduling processes, and handling threads. These things themselves affect execution efficiency, and different OS handle them differently and also affect the performance. Libraries themselves provide optimized algorithms. Different libraries worked differently. For example, there are static libraries and dynamic libraries. Using the correct libraries can enable programs to take specific advantages and improve performance. In general, software determines how efficiently hardware is used, and software choices can lead to large variations in runtime even on the same hardware. Presenting the software configuration is important for interpreting the results.

The system configuration, especially the state of the system during execution affects the accuracy and consistency of performance measurements. Since systems run multiple processes simultaneously, the CPU time, cache, and memory resources are shared. The number of background programs should be minimalized since background applications can reduce the resources available to the program being measured, which would result in slower and more unstable execution times. Cache interference can increase memory access delays, as caches are shared among programs and other programs can overwrite caches. This would result in the program to reload the data and thus slows the performance. System load can also cause inconsistent results since if system load changes then available CPU changes, which would result in the runtime to vary. Because of these sources of variability, performance measurements can contain noise, making it necessary to control system conditions, minimize background activity, and perform multiple runs with averaging. In general, properly reporting system state ensures that results are reliable and comparable.

In general, these 3 configurations ensure 3 things. Hardware configuration defines the system's capabilities, software configuration determines how efficiently those capabilities are used, and system state influences the accuracy of measurements. They had to be properly presented with the performance results so the raw data can be transformed into valuable results that could guide the design decisions and system improvements.

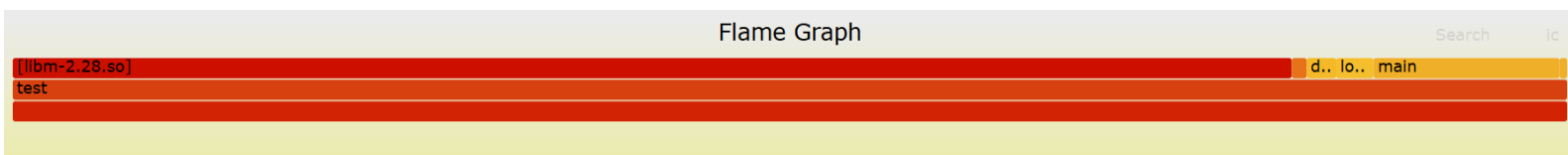
Carl Chang:

For day 2:

I discussed how to use flame graphs to show performance results.

Here's what I did. Firstly I presented a step by step instruction on how to install the files and create a flame graph from a program. Then I showed a few flame graphs that I created from a few programs and used them to explain how to read flame graphs. Basically, the width of each red bar represents the amount of time and resources it consumes. The smaller bars above the bigger bars represent the sub parts of a program or a function. The first flame graph was from a really simple code. It showed that the program ran properly without parts that were consuming significantly more times than others. This was due to the fact that the compiler on Sunlab had done proper optimization to the program at compilation. In order to show how the flame graph showed the problem of the program, the second flame graph was created from a program that was purposely made to not be properly optimized. The graph showed that some parts of the program had consumed significantly more times than the others. Lastly I showed the flame graph I created from the code from P1 that used TBB parallel with count sort. An interesting thing here was that the main() showed up separately from the parallel work, which meant that it was possible that the main function is spin waiting or TBB distributes work across working threads. It was really important to figure out the actual reasons for results like this.

Below were the graphs I used:



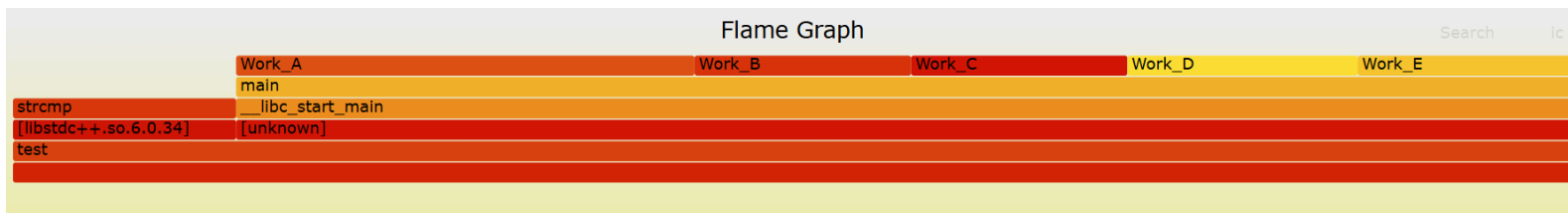
The graph above was the I graph I created from a really simple code with a few separated functions. There weren't parts that consumed significantly more times than others because of the compiler optimization.

Below is part of the codes:

```

5  const int ITER = 100000000;
6
7  void Work_A() {
8      volatile int x = 0;
9      for (int i = 0; i < ITER; i++){
10         x += i;
11     }
12 }
13
14 void Work_B() {
15     volatile int x = 0;
16     for (int i = 0; i < ITER; i++){
17         x += i * 2;
18     }
19 }
20
21 void Work_C() {
22     volatile int x = 0;
23     for (int i = 0; i < ITER; i++){
24         x += i * 3;
25     }
26 }
27
28 void Work_D() {
29     volatile int x = 0;
30     for (int i = 0; i < ITER; i++){
31         x += i * 4;
32     }
33 }
34
35 void Work_E() {
36     volatile int x = 0;
37     for (int i = 0; i < ITER; i++){
38         x += i * 5;
39     }
40 }
41
42 int main() {
43     Work_A();
44     Work_B();
45     Work_C();
46     Work_D();
47     Work_E();
48 }

```



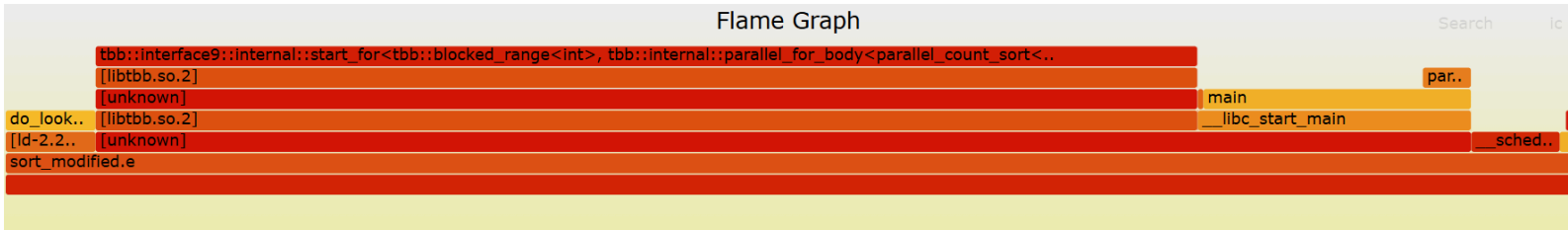
The graph above was created from a slightly modified version of the previous code. The code here was purposely modified to not have proper optimization at compilation. As shown in the graph, some functions were taking more time than the others and also all of them were in separate blocks.

Below's part of the codes:

```

5  const int ITER = 100000000;
6
7  __attribute__((noinline))
8  void Work_A() {
9      volatile int x = 0;
10     for (int i = 0; i < ITER; i++){
11         x += i;
12     }
13 }
14
15 __attribute__((noinline))
16 void Work_B() {
17     volatile int x = 0;
18     for (int i = 0; i < ITER; i++){
19         x += i * 2;
20     }
21 }
22
23 __attribute__((noinline))
24 void Work_C() {
25     volatile int x = 0;
26     for (int i = 0; i < ITER; i++){
27         x += i * 3;
28     }
29 }
30
31 __attribute__((noinline))
32 void Work_D() {
33     volatile int x = 0;
34     for (int i = 0; i < ITER; i++){
35         x += i * 4;
36     }
37 }
38
39 __attribute__((noinline))
40 void Work_E() {
41     volatile int x = 0;
42     for (int i = 0; i < ITER; i++){
43         x += i * 5;
44     }
45 }
46
47 int main() {
48     Work_A();
49     Work_B();
50     Work_C();
51     Work_D();
52     Work_E();
53 }

```



The graph above was the last graph I used that was created from the TBB parallel count sort code I used for P1 (by the way, this was the fastest code I had for P1). Notice that the main() showed up separately from the parallel work. This might be that it was spring waiting or TBB distributes work across threads.

Section: Performance Benchmarking Rigour: Analysis of System Behaviour and Topology

Prepared by: Rittambhra Rani

During our presentation on parallel performance evaluation, my segments concentrated on identifying systemic measurement errors, frequently termed "benchmarking crimes", alongside demonstrating how high-dimensional data visualisation can illuminate underlying hardware topology. Rather than simply reviewing superficial graphing rules, the objective was to examine how specific measurement tools and visualisation strategies fundamentally mask or reveal authentic system behaviour.

1. Benchmarking Crime #1: Coordinated Omission

A critical oversight within systems benchmarking is Coordinated Omission, a phenomenon comprehensively outlined by Gil Tene. Standard load generators, such as ApacheBench or JMeter, typically operate on a closed-loop system where they must wait for a response before issuing the subsequent request. Consequently, if the System Under Test (SUT) experiences a severe stall, such as a ten-second Garbage Collection pause, the benchmarking tool pauses concurrently.

Because the tool ceases to send requests during this failure state, it effectively takes zero samples of the worst-case behaviour. This dynamic introduces severe survivor bias into the dataset. The resulting percentile distribution curve will inevitably display a false sense of stability, reporting artificially low tail latencies at the 99th and 99.9th percentiles. To accurately measure real-world performance and capture these transient stalls, engineers must evaluate systems using open-loop generators that maintain a constant arrival rate irrespective of the server's immediate health.

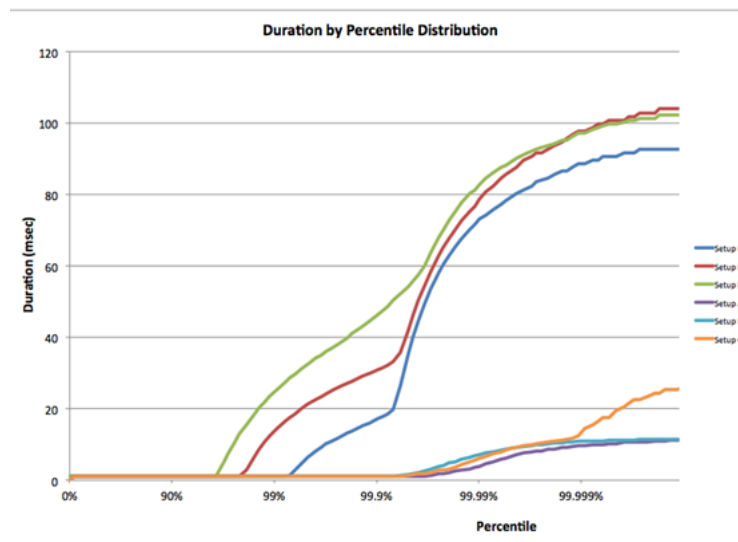


Fig. 1: The Gil Tene "Coordinated Omission" percentile graph. From: https://groups.google.com/g/mechanical-sympathy/c/icNZJejUHfE/m/BfDekfBEs_sJ?pli=1

2. Benchmarking Crime #2: The Throughput-Latency Decoupling Fallacy

The second measurement fallacy discussed was the erroneous treatment of peak throughput and minimum latency as independent metrics. In reality, these two values are inversely correlated due to the fundamental principles of queueing theory. As dictated by Little's Law ($L = \lambda W$), when system utilisation approaches 100%, queue lengths and their associated latencies approach infinity.

Reporting maximum throughput alongside minimum latency as isolated figures frequently obscures the fact that high throughput is typically achieved via heavy buffering and batching.

These structural techniques inherently degrade latency. To present honest limits regarding system capacity, engineers must plot the full saturation curve, colloquially known as the "hockey stick" graph. This visual representation explicitly identifies the "knee" of the curve, denoting the precise load threshold where the system reaches saturation and latency begins to degrade non-linearly.

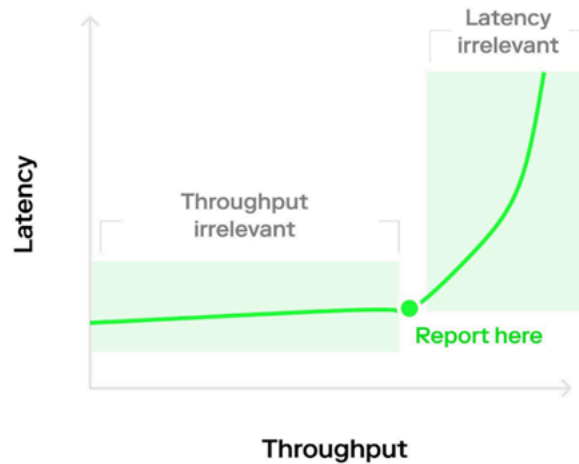


Fig. 2: The Saturation Curve (or "Hockey Stick") graph. From: <https://en.theblockbeats.news/news/32607>

3. Visualising Micro-Architectural Topology via Heat Maps

When analysing high-dimensional data, such as core-to-core latency across a modern multi-core processor, traditional line graphs degenerate into unreadable "spaghetti plots" that completely obscure the spatial relationships of the hardware. To resolve this limitation, I presented the application of heat maps for topological visualisation.

By mapping latency to a colour gradient upon a grid, the heat map effectively functions as a physical schematic of the silicon die. This method immediately surfaces the "block diagonal" structure of the architecture:

- **Small Blocks (2 * 2):** These visually isolate Simultaneous Multithreading (SMT) siblings sharing L1 and L2 cache, demonstrating near-zero latency.
- **Large Blocks (4 * 4):** These regions reveal the Core Complex (CCX) boundaries, grouping physical cores that share a unified L3 cache.
- **Off-Diagonal Zones:** The distinct high-latency areas instantly expose the interconnect penalty incurred when data must travel across the chip's Infinity Fabric.

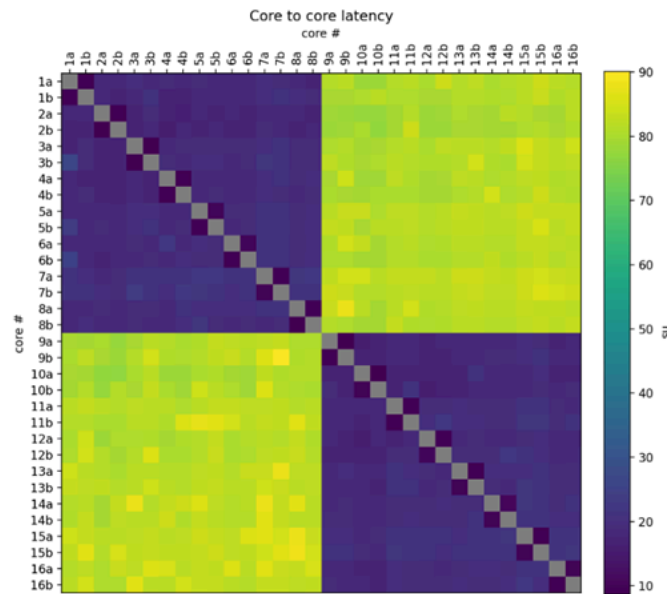


Fig. 3: The Core-to-Core Latency Heat Map. From <https://github.com/andportnoy/core-to-core-latency>

Ultimately, this approach represents far more than mere data visualisation; it serves as a highly actionable engineering tool. Grasping this hardware topology is precisely what allows developers to write effective thread affinity policies (`sched_setaffinity`), ensuring that communicating threads remain pinned to the same CCX domain to prevent severe interconnect latency penalties.

Santhoshi Karri (sak724@lehigh.edu)
Presentation Report

Concept: Generalizing Latency, Throughput, and Bandwidth Beyond Networking

The original presentation described **latency**, **throughput**, and **bandwidth** mostly in terms of network behavior. I expanded this section because these metrics are broader **software performance engineering** concepts, not just networking terms. In a performance engineering context, **latency** means the time required for one operation to complete, **throughput** means the number of operations completed per unit of time, and **bandwidth** means the rate at which data can be moved through a resource. These definitions apply to many parts of a system, including **data structures, memory, disk, the operating system, and the file system**.

To make this concrete, I implemented **microbenchmarks** instead of only giving definitions. For data structures, I measured lookup performance on **std::vector**, **std::unordered_map**, and **std::map**. Each structure was tested in two modes. In **latency mode**, each lookup depends on the result of the previous lookup, creating a **dependency chain**. This measures the cost of a single operation when the next operation cannot proceed until the current one finishes. In **throughput mode**, the lookup keys are independent, so the benchmark measures how many lookup operations can be completed per second. This shows that the same operation, a lookup, can measure different things depending on how the benchmark is written.

The results show the distinction clearly. For **std::vector**, lookup latency was about **9.53 ns per lookup**, while throughput mode reached about **1.03 billion lookups per second**. For **std::unordered_map**, latency was about **118.40 ns per lookup**, while throughput mode reached about **63.48 million lookups per second**. For **std::map**, latency was about **387.21 ns per lookup**, while throughput mode reached about **2.74 million lookups per second**. **std::map** was slower than the other structures because tree lookup involves **pointer chasing** and **branch-heavy traversal**. These results demonstrate that latency and throughput can be measured on local data-structure operations, not only on packets moving across a network.

I also added **system-level benchmarks** to address bandwidth and system-call latency. **Memory bandwidth** was measured by copying a **512 MB** buffer five times, transferring **2.68435 GB** total and achieving about **22.40 GB/s**. **Disk write bandwidth** was measured by writing a **64 MB** file to scratch storage using **1024 KB** blocks, achieving about **0.068 GB/s**. **System-call latency** was measured by calling **getpid()** **10,000,000** times, which produced an average latency of about **66.45 ns per system call**. I also measured **file-system throughput** by creating **5,000** small files, which achieved about **960.40 files per second**. These experiments show that bandwidth can refer to **memory bandwidth** or **disk bandwidth**, throughput can refer to **file-system operations**, and latency can refer to **operating-system operations** such as system calls.

Overall, this revision makes the topic more accurate for **software performance engineering**. The key point is that a metric name by itself is not enough. We must state **what operation is being measured** and **which system resource is involved**. Latency may describe a **data-structure lookup** or a **system call**. Throughput may describe **lookups per second** or **files created per second**. Bandwidth may describe **memory transfer** or **disk write rate**. This makes the presentation more precise and avoids treating performance metrics as if they only apply to networking.

Concept	Benchmark	Result
Vector Latency	Dependent vector lookup	9.53 ns/lookup
Vector Throughput	Independent vector lookup	1.03B lookups/s
Hash-Table Latency	Dependent hash lookup	118.40 ns/lookup
Hash-Table Throughput	Independent hash lookup	63.48M lookups/s
Tree Latency	Dependent tree lookup	387.21 ns/lookup
Tree Throughput	Independent tree lookup	2.74M lookups/s
Memory Bandwidth	512 MB copy, 5 passes	22.40 GB/s
System-Call Latency	10M getpid() calls	66.45 ns/syscall
Disk Bandwidth	64 MB write	0.068 GB/s
File-System Throughput	5,000 file creations	960.40 files/s

Concept: How to Read a Flame Graph and Flame Graph Analysis

A flame graph is not a timeline. It does not show the order in which functions are executed. Instead, it provides an aggregate view of where the program was spending time while it was running. Each sample records the current call stack, and the flame graph combines many sampled call stacks into one visualization. This makes it useful for identifying hot paths in a program.

The most important feature of a flame graph is box width. A wider box means that the function appeared in more samples, so it represents a larger share of the observed execution time. The vertical position shows the calling context. Functions lower in the graph are callers, and functions above them were called from those paths. A function high in the graph is not automatically more expensive; it is simply deeper in the call stack. A narrow box high in the graph may be less important than a wide box lower in the graph.

Flame graphs are useful because they preserve calling context. Instead of only showing that a function is expensive, they show which path led to that function. This matters when the same function is called from multiple parts of a program. If a function appears in multiple towers, it may not appear as one single wide box. In that case, searching the SVG file or the folded stack file helps locate all occurrences of that function across the profile.

For this experiment, I created a separate workload in `flamegraph.cpp` with three lookup paths: vector lookup, hash-table lookup, and tree lookup. This made the profiling example self-contained and directly connected it to the data-structure experiments. I generated the flame graph using `perf` and `FlameGraph` tooling, which produced both an SVG flame graph and a folded stack file. The SVG provides the visual profile, while the folded stack file provides the exact stack paths in searchable text form.

The main question was whether a function that appears in multiple towers can be found. To test this, I searched the folded stack file for the shared function `normalize_key`. The output showed `normalize_key` in two different stack paths: `process_hash_path` $\rightarrow$ `hash_lookup` $\rightarrow$ `normalize_key` and `process_vector_path` $\rightarrow$ `vector_lookup` $\rightarrow$ `normalize_key`. The command `grep "normalize_key" results/out.folded | wc -l` returned 2, confirming that the function appeared in multiple stack entries. This demonstrates that repeated functions in a flame graph can be located either visually in the SVG or textually by searching the folded stack file.

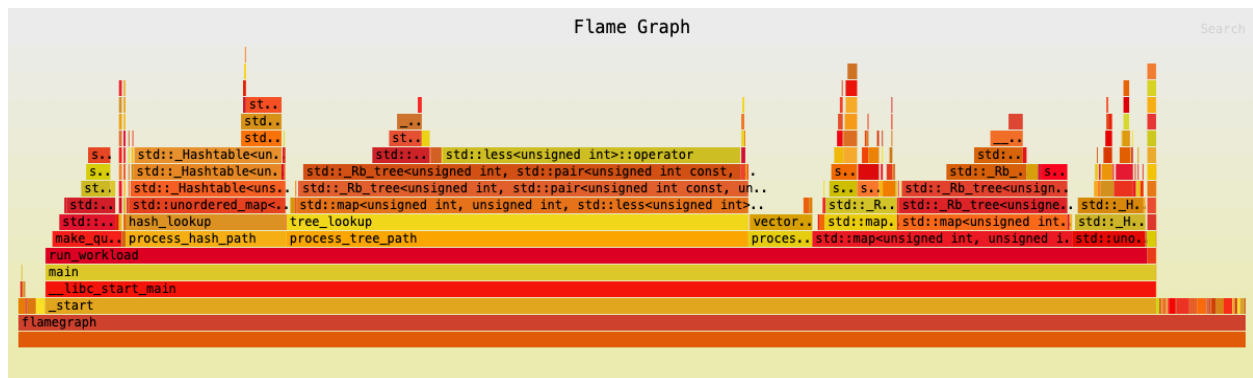


Figure: Flame graph of the lookup workload.

The flame graph shows sampled stack traces from the flamegraph.cpp workload. The main visible paths include `process_hash_path`, `hash_lookup`, `process_tree_path`, `tree_lookup`, `std::unordered_map`, and `std::map` internals. Wider boxes indicate functions that appeared more frequently in the samples. This figure supports the folded-stack search result by showing the separate lookup paths that were sampled during execution.

Concept: CPU Time vs Elapsed Time

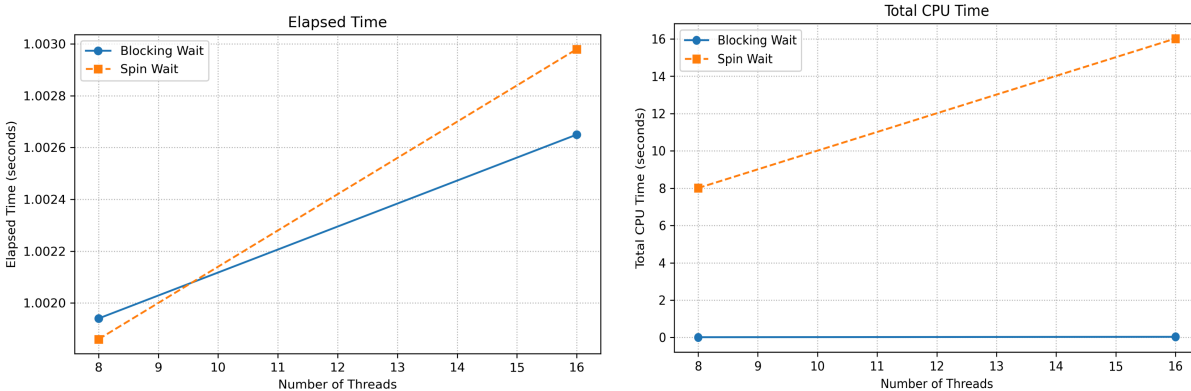
In the paper “How to Measure, Present, and Compare Parallel Performance,” Lawrence A. Crowl explains that elapsed time is usually the most meaningful measurement for parallel performance because it represents the total time required to finish the computation. Elapsed time includes both useful computation and time spent waiting for synchronization, locks, scheduling, paging, or I/O.

CPU time is different: it measures how much processor time the program consumed while actively executing.

To demonstrate this idea, I implemented a small parallel workload with two waiting strategies. In the blocking-wait version, worker threads wait using operating-system synchronization. While they are waiting, the operating system can put those threads to sleep, so they contribute very little CPU time. In the spin-wait version, worker threads repeatedly check an atomic flag until they are released. These threads are still waiting, but they remain active on the CPU, so they consume processor time even though they are not making useful progress.

This experiment shows why CPU time and elapsed time must be interpreted separately. Both workloads have almost the same elapsed time because the main thread delays the worker release by the same fixed amount of time. However, the CPU time is very different. Blocking wait has low CPU time because the waiting threads sleep. Spin wait has high CPU time because the waiting threads burn CPU cycles while polling. As the number of spinning threads increases, total CPU time also increases, even though elapsed time stays almost unchanged.

This supports Crawl's argument that elapsed time measures the real time to solution, while CPU time helps diagnose how processor resources were used. A low CPU time does not always mean the program finished quickly; it may mean the program spent time sleeping or waiting. A high CPU time does not always mean useful work was done; it may mean the program wasted processor cycles through spinning. For parallel performance reporting, elapsed time should be reported because it reflects what the user experiences, while CPU time can help explain whether the program was computing, sleeping, or spinning.



The results support the expected behavior. In the blocking-wait case, elapsed time is close to one second while CPU time remains very small. In the spin-wait case, elapsed time is still close to one second, but CPU time increases roughly with the number of threads. This confirms that the two cases have similar time-to-completion but very different processor usage.

Concept: Baseline Selection and Avoiding Speedup Distortion

In “How to Measure, Present, and Compare Parallel Performance,” Lawrence A. Crowl explains that speedup can be misleading because it depends on the chosen sequential baseline. A parallel program running with one thread is not necessarily the same as a true sequential program. It may still include thread creation, joining, synchronization, or other parallel-implementation overheads. For this reason, a fair comparison should identify the baseline explicitly instead of assuming that “parallel with one thread” is equivalent to a sequential implementation.

To demonstrate this, I implemented a small workload with a true sequential version and a parallel version run with different thread counts. The true sequential baseline took 0.112177 seconds, while the parallel implementation with one thread took 0.110038 seconds. These values are close, but they come from different implementations. The result shows why the baseline must be documented clearly: even small differences in baseline choice can affect any derived speedup number. Instead of using speedup as the main result, I presented elapsed time directly, which is consistent with the paper’s recommendation to avoid unnecessary derived metrics.

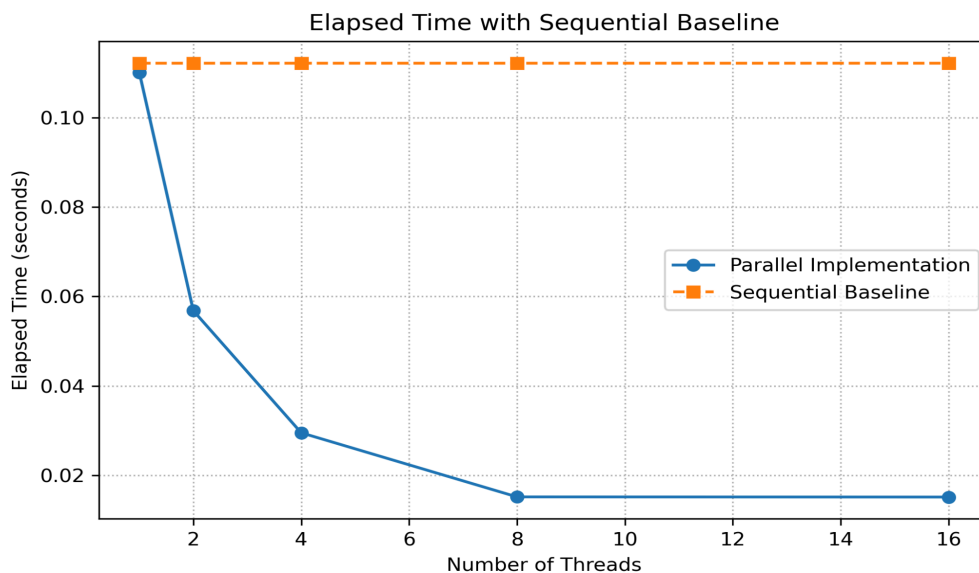


Figure: Elapsed Time with Sequential Baseline.

This figure compares the elapsed time of the parallel implementation against the true sequential baseline. The dashed horizontal line represents the single-thread sequential implementation and is repeated across thread counts only as a reference. The plot avoids presenting speedup directly and instead shows elapsed time, which follows the recommendation in “How to Measure, Present, and Compare Parallel Performance.”

Concept: Variance and Scalability Instability

In “How to Measure, Present, and Compare Parallel Performance,” Lawrence A. Crawl argues that execution time should not be measured only once because parallel programs are affected by scheduling, synchronization, memory behavior, and system noise. Measuring only the average can hide instability. Variance is important because high run-to-run variation may indicate contention, synchronization pressure, or unstable scalability.

To demonstrate this, I implemented a repeated lock-contention experiment using two synchronization mechanisms: a test-and-set spin lock and a standard mutex. Each configuration was run ten times for different thread counts. I recorded the mean execution time, standard deviation, minimum, maximum, and coefficient of variation. The coefficient of variation is useful because it shows variability relative to the mean, making it easier to compare instability across thread counts.

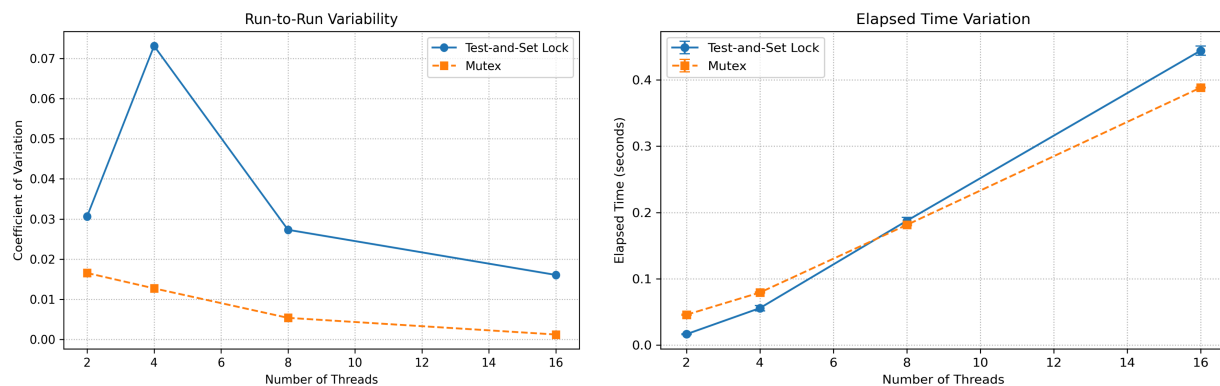


Figure: Elapsed Time and Run-to-Run Variability Under Lock Contention.

These figures compare elapsed time and run-to-run variability for a test-and-set lock and a mutex. The elapsed-time plot shows that execution time increases as the number of threads grows, which indicates increasing contention. The variability plot shows that the test-and-set lock has higher coefficient of variation than the mutex, especially at 4 threads. This means the spin lock produced less stable execution times across repeated runs. The mutex was more consistent because waiting threads can be managed by the operating system instead of continuously spinning. Together, the figures show why variance should be reported: average elapsed time shows performance cost, while variability reveals instability that a single runtime number would hide.

Concept: Log-Log Time Scaling

In “How to Measure, Present, and Compare Parallel Performance,” Lawrence A. Crowl explains that the way performance data is plotted can change how the reader interprets scaling behavior. A linear time plot shows absolute elapsed time directly, but it can make scaling trends harder to judge when the number of processors or threads changes by factors. A log-log time plot gives relative changes more balanced visual space, which makes it easier to compare measured performance against ideal scaling and identify where improvement begins to flatten.

For this experiment, I used the elapsed-time data from the baseline experiment and plotted it in two forms: a linear time plot and a log-log time plot. Both figures show the same measured data and the same ideal linear-scaling reference. In both plots, lower elapsed time is better. The measured implementation improves from 1 to 8 threads, but the improvement becomes limited between 8 and 16 threads. The log-log plot makes this flattening easier to see relative to the ideal scaling line.

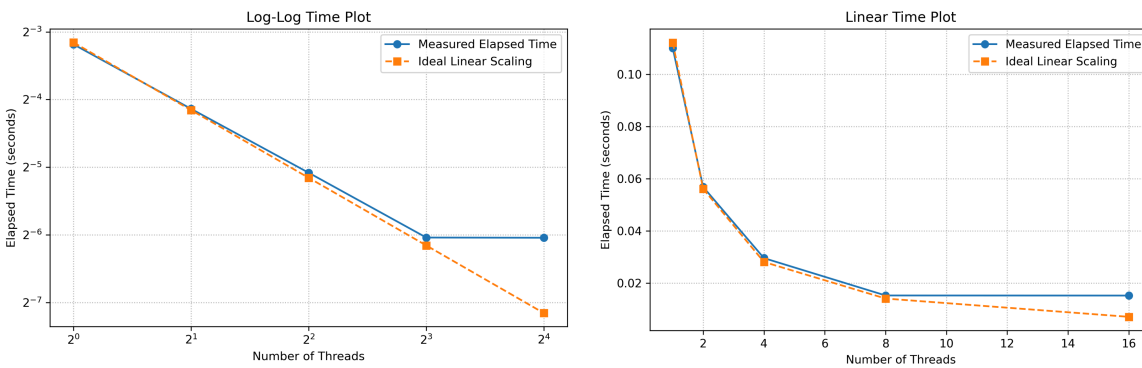


Figure: Linear and Log-Log Time Scaling.

These figures show the same elapsed-time data using two different graph scales. The linear plot shows the absolute decrease in runtime as the number of threads increases. The log-log plot shows the same trend using logarithmic axes, which makes relative scaling behavior easier to compare. In both figures, lower elapsed time is better. The measured program improves as the thread count increases, but the improvement flattens after 8 threads. This supports the paper’s point that log-log time plots can make scaling behavior and breakdown points easier to identify.